



EE/CE 252/251 Signals and Systems

Spring 2026

Milestone 1 Report

Team Members:

Maaz Haris(mh09633), Rauf Abdullah(ra10307), & Basil Saeed

Bari(bb09892)

Instructor:

Mr.Tariq Mumtaz



Contents

1	Introduction	3
1.1	Project Objective	3
1.2	Methodology	3
2	Tasks	4
2.1	Duration of the Recorded Signals (in seconds)	4
2.1.1	Code	4
2.1.2	Results	4
2.2	Keynote and Overtone frequencies of the signals	4
2.2.1	Code	5
2.2.2	Results and Plots	8
2.3	Signal Corruption and Regeneration	11
2.3.1	Cello: Corruption and Partial Recovery	11
2.3.2	Code (Cello)	12
2.3.3	Results and Plots (Cello)	14
2.3.4	Trombone: Corruption and Full Recovery	15
2.3.5	Code (Trombone)	15
2.3.6	Results and Plots (Trombone)	17

List of Figures

1	Cello Frequencies (Linear)	8
2	Cello Frequencies (logarithmic)	8
3	Trombone Frequency (Linear)	9
4	Trombone Frequencies (logarithmic)	10
5	Cello Partial Recovery: Demonstration of Lost Samples	14
6	Cello Recovery via Spline Interpolation (First 150 Samples)	14
7	Verification of Complete Trombone Recovery	17

1 Introduction

1.1 Project Objective

The primary purpose of this project is to study and solve real-life engineering problems related to Signals and Systems using both real measurements and simulated effects. Acoustic instruments produce sound that consists of a fundamental frequency, known as the keynote, along with a series of overtones. By analyzing tones played by instruments such as the cello and trombone, alongside three additional user-selected instruments, this project investigates the distinct frequency characteristics that define each instrument's unique sound. Furthermore, the project explores signal corruption and data recovery, testing the feasibility of restoring original audio data after it has been distorted by faulty recording system components.

1.2 Methodology

To achieve these objectives, MATLAB was utilized as the primary computational tool for signal analysis. The investigation began by importing time-domain audio samples and utilizing spectral analysis to estimate their frequency content. By transitioning the data from the time domain to the frequency domain, the fundamental keynotes and their corresponding overtones could be isolated and examined on both linear and logarithmic scales. This frequency-domain analysis was critical for identifying precise overtone offsets caused by physical constraints like string stiffness. For the signal recovery milestones, mathematical models of the corrupting systems—including downsampling and moving average filters—were analyzed to formulate and code appropriate inverse difference equations aimed at retrieving the original data. Through this systematic methodology, complex problems in signal processing were investigated and resolved.

2 Tasks

2.1 Duration of the Recorded Signals (in seconds)

2.1.1 Code

In this we find the total number of samples in each sample, cello and trombone and multiplied it with period of the signal to find the duration of the signal as the duration of a signal is given by the following formulas

$$t = \frac{\text{Total Samples}}{\text{Sampling Rate}}$$
$$t = \text{Total Samples} \times \text{Sampling Period}$$

```
1      %Calculating the number of samples in cello and trombone
2      Nc = length(cello.x)
3      Nt = length(trombone.x)
4
5      %Calculating the time of each instrument
6      Tc = Nc * cello.dt
7      Tt = Nt * trombone.dt
8
9      fprintf('Length of Cello recording: %.4f seconds\n', Tc);
10     fprintf('Length of Trombone recording: %.4f seconds\n', Tt);
```

2.1.2 Results

Using the above formula and code we found out that the duration of both cello and trombone sounds was 1.338 seconds long.

2.2 Keynote and Overtone frequencies of the signals

The acoustic signature of an instrument is defined by its harmonic series, which consists of a fundamental frequency and subsequent overtones. The fundamental frequency, or

keynote (f_0), is the lowest frequency in the harmonic series and determines the perceived pitch of the sound. Overtones are the higher frequencies that accompany the keynote. The relationship between their amplitudes and phases produces the unique timbre of the instrument. Ideally, for string and wind instruments, these overtones appear at exact integer multiples of the fundamental frequency, modeled by the equation:

$$f_k = kf_0, \quad k = 1, 2, \dots$$

2.2.1 Code

In this part we first calculate the sampling frequency of the signal (F_s) measured in Hertz(Hz). As we are given the sampling period in the data, which is the time between each recorded data point, the sampling frequency is the inverse of the sampling period $F_s = \frac{1}{T_s}$. Then we created a frequency vector from the 0 Hz to the the Nyquist frequency ($F_s/2$) based on the number of samples. Then we used MATLAB's builtin fft function to convert the time domain signal into a frequency domain signal. This function gives me a vector containing complex numbers that encode both magnitude and phase information of each frequency present in the signal. As the result of fft signal is based on the length on the input signal, we normalized the results to reflect the true physical amplitudes. As fft gives us a two sided spectrum representing both negative and positive frequencies, and negative frequencies have physical meaning, so the second half of the spectrum is removed, but we make sure that the energy of the signal and amplitudes are unchanged. Then we plot the results of this on two different scales, Linear and Logarithmic.

```
1      % --- Cello FFT ---
2      % Calculating the Sampling frequency
3      Fs_cello = 1 / cello.dt;
4
5      %Creating a frequency vector for x axis
6      f_cello = Fs_cello * (0:(Nc/2)) / Nc;
7
8      % Computing FFT(Fast Fourier Transform)
9      Y_cello = fft(cello.x);
```

```
10
11     %Normalizing the signal and considering magnitude of the signal
12     P2_cello = abs(Y_cello / Nc);
13
14     %Converting the two sided signal into a single sided signal and
    making sure
15     %that the energy if the signal is conserved
16     P1_cello = P2_cello(1:Nc/2+1);
17     P1_cello(2:end-1) = 2 * P1_cello(2:end-1);
18
19     % --- Trombone FFT ---
20     % Calculating the Sampling frequency
21     Fs_trombone = 1 / trombone.dt;
22
23     %Creating a frequency vector for x axis
24     f_trombone = Fs_trombone * (0:(Nt/2)) / Nt;
25
26     % Computing FFT(Fast Fourier Transform)
27     Y_trombone = fft(trombone.x);
28
29     %Normalizing the signal and considering magnitude of the signal
30     P2_trombone = abs(Y_trombone / Nt);
31
32     %Converting the two sided signal into a single sided signal and
    making sure
33     %that the energy if the signal is conserved
34     P1_trombone = P2_trombone(1:Nt/2+1);
35     P1_trombone(2:end-1) = 2 * P1_trombone(2:end-1);
36
37     % --- Plotting Cello (Linear & Logarithmic) ---
38     figure('Name', 'Cello Frequency Spectrum');
39     plot(f_cello, P1_cello);
40     title('Cello Frequency Spectrum (Linear)');
41     xlabel('Frequency (Hz)'); ylabel('Magnitude');
42     xlim([0 3000]); % Zoom in to see the peaks clearly
```

```
43     grid on;
44
45     figure;
46     semilogy(f_cello, P1_cello);
47     title('Cello Frequency Spectrum (Logarithmic)');
48     xlabel('Frequency (Hz)'); ylabel('Magnitude (Log)');
49     xlim([0 3000]);
50     grid on;
51
52
53     % --- Plotting Trombone (Linear & Logarithmic) ---
54     figure('Name', 'Trombone Frequency Spectrum');
55     plot(f_trombone, P1_trombone);
56     title('Trombone Frequency Spectrum (Linear)');
57     xlabel('Frequency (Hz)'); ylabel('Magnitude');
58     xlim([0 3000]);
59     grid on;
60
61     figure
62     semilogy(f_trombone, P1_trombone);
63     title('Trombone Frequency Spectrum (Logarithmic)');
64     xlabel('Frequency (Hz)'); ylabel('Magnitude (Log)');
65     xlim([0 3000]);
66     grid on;
67
```

2.2.2 Results and Plots

Cello

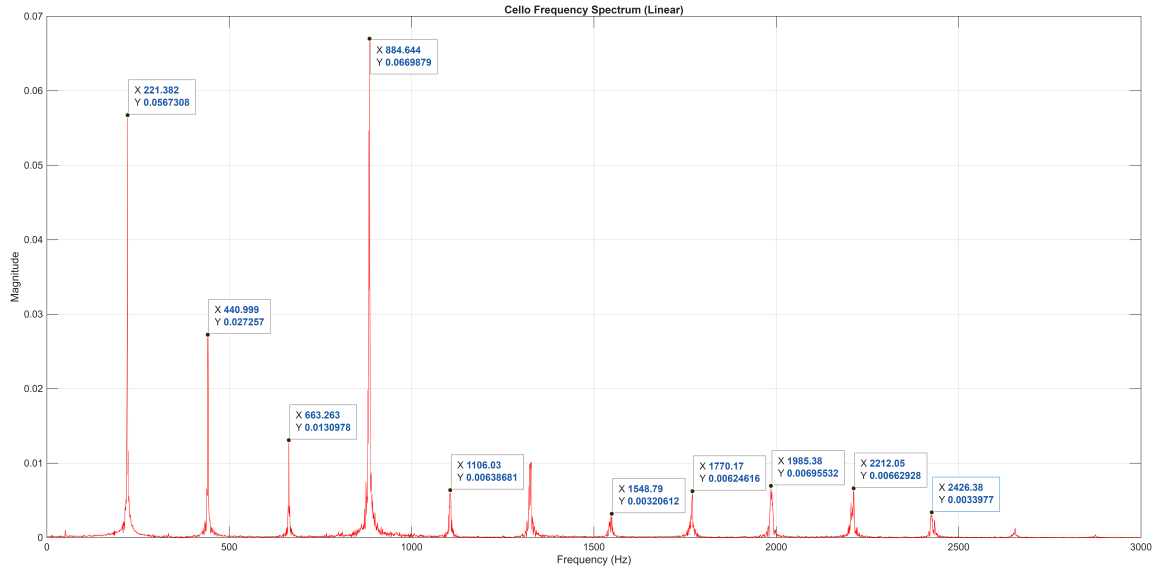


Figure 1: Cello Frequencies (Linear)



Figure 2: Cello Frequencies (logarithmic)

- **Keynote Frequency:** As observed in the Linear Frequency Spectrum (Figure 1), the keynote frequency of Cello is 221.382 Hz
- **Overtone:** Using the formula $f_k = k f_0$, we found that the predicted overtone frequencies of the cello signal are $f_1 = 442.764 \text{ Hz}$, $f_2 = 664.146 \text{ Hz}$, $f_3 = 885.528$.

However the overtones found using the Linear Frequency Spectrum (Figure 1), we found that the overtones are $f_1 = 440.999 \text{ Hz}$, $f_2 = 663.262 \text{ Hz}$, $f_3 = 884.644 \text{ Hz}$. As we can see that the the overtone frequencies observed are not the exact multiple of the keynote frequency. This offset is due to the stiffness of the string and the model of the overtones taking stiffness into account is

$$f_k = k f_0 \sqrt{1 + B k^2}$$

(where B is the positive stiffness parameter).

- **Number of Overtones:** As Overtones carry significantly less energy, the Logarithmic Frequency Spectrum is required to accurately count them. Using the logarithmic Spectrum of Cello (Figure 2), there are 12 distinct overtones clearly visible above the noise floor.

Trombone

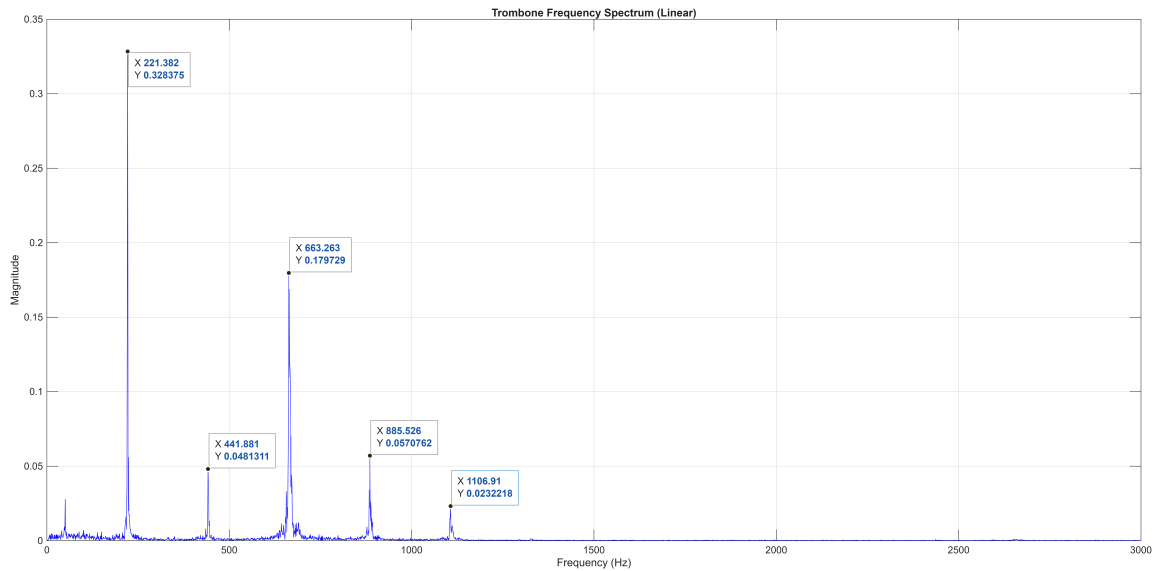


Figure 3: Trombone Frequency (Linear)

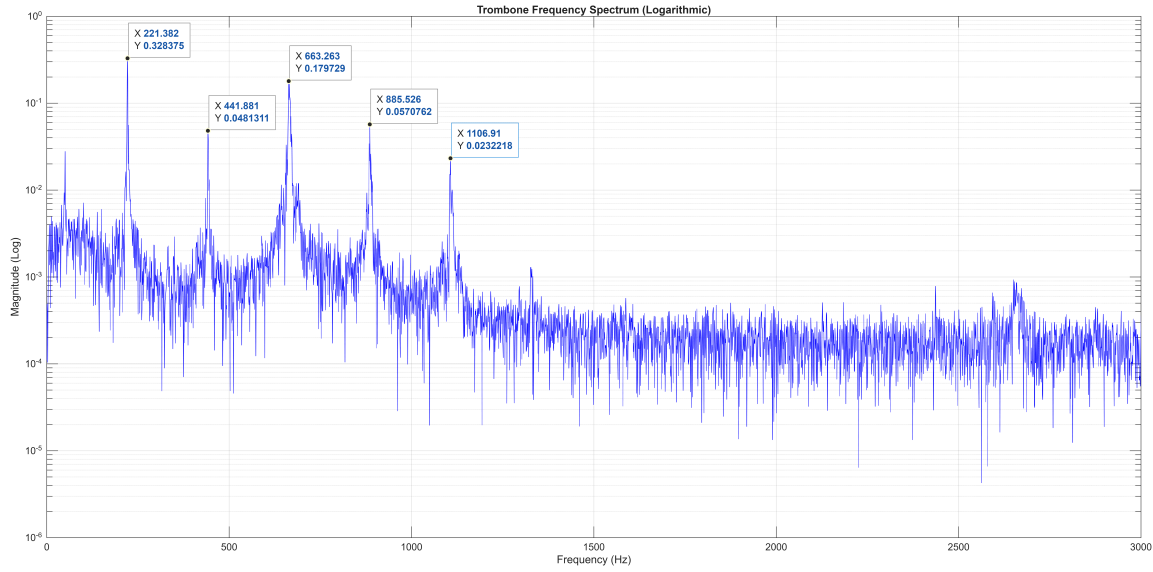


Figure 4: Trombone Frequencies (logarithmic)

- **Keynote Frequency:** As observed in the Linear Frequency Spectrum (Figure 3), the keynote frequency of Cello is 221.382 Hz
- **Overtone:** Using the formula $f_k = kf_0$, we found that the predicted overtone frequencies of the cello signal are $f_1 = 442.764 Hz$, $f_2 = 664.146 Hz$, $f_3 = 885.528$. However the overtones found using the Linear Frequency Spectrum (Figure 3), we found that the overtones are $f_1 = 441.881 Hz$, $f_2 = 663.262 Hz$, $f_3 = 885.526 Hz$. As we can see that the overtone frequencies observed are not the exact multiple of the keynote frequency. This offset is due to the stiffness of the string and the model of the overtones taking stiffness into account is

$$f_k = kf_0\sqrt{1 + Bk^2}$$

(where B is the positive stiffness parameter)

- **Number of Overtones:** As Overtones carry significantly less energy, the Logarithmic Frequency Spectrum is required to accurately count them. Using the logarithmic Spectrum of Cello (Figure 4), there are 4 distinct overtones clearly visible above the noise floor.

2.3 Signal Corruption and Regeneration

This section investigates two distinct corrupting systems applied to the cello and trombone signals respectively. For each instrument, the corrupting system is modeled mathematically, an inverse or recovery procedure is derived, and the result is implemented and verified in MATLAB.

2.3.1 Cello: Corruption and Partial Recovery

The cello signal $x_c[n]$ is corrupted according to the formula

$$y_c[n] = n \cdot x_c[2n + 1] + 5, \quad n = 1, 2, \dots, N_{\max}$$

where $N_{\max} = \lfloor (\text{length}(x_c) - 1)/2 \rfloor$. This corruption combines three operations: (i) downsampling, since only the odd-indexed samples $x_c[3], x_c[5], x_c[7], \dots$ are retained and all even-indexed samples are permanently discarded; (ii) time-varying amplitude scaling, where sample n is multiplied by its index n ; and (iii) a constant DC offset of +5 added to every sample.

Because the even-indexed samples are irretrievably lost, full recovery of the original signal is impossible. However, the retained odd-indexed samples can be exactly restored by algebraically inverting the scaling and DC offset. Rearranging the corruption formula gives the partial recovery equation:

$$\hat{x}_c[2n + 1] = \frac{y_c[n] - 5}{n}, \quad n = 1, 2, \dots, N_{\max}$$

The even-indexed positions remain unknown and are set to zero in the reconstructed timeline. To estimate these missing samples, cubic spline interpolation is applied over the known odd-indexed values. Spline interpolation is preferred over linear interpolation for audio signals as it produces a smooth, differentiable curve that better approximates the underlying continuous waveform.

2.3.2 Code (Cello)

```
1 % =====
2 % 6. CELLO CORRUPTION & PARTIAL RECOVERY
3 % =====
4 % Formula: CelloCorrupted[n] = n * cello[2n + 1] + 5
5 N_cello_max = floor((length(cello.x) - 1) / 2);
6 cello_corrupted = zeros(N_cello_max, 1);
7
8 % 6a. Simulate corruption
9 for n = 1:N_cello_max
10     cello_corrupted(n) = n * cello.x(2*n + 1) + 5;
11 end
12
13 % 6b & 6c. Cello Partial Recovery
14 cello_recovered_partial = zeros(N_cello_max, 1);
15 for n = 1:N_cello_max
16     cello_recovered_partial(n) = (cello_corrupted(n) - 5) / n;
17 end
18
19 % --- VISUALIZATION: Reconstructing the timeline ---
20 % Map the recovered odd samples back to their original timeline
    positions.
21 % The lost even samples will remain as 0.
22 cello_full_timeline = zeros(length(cello.x), 1);
23 for n = 1:N_cello_max
24     cello_full_timeline(2*n + 1) = cello_recovered_partial(n);
25 end
26
27 % Plotting Original vs. Partially Recovered Cello
28 figure('Name', 'Cello Partial Recovery Verification');
29 stem(1:100, cello.x(1:100), 'b', 'marker', 'none'); hold on;
30 stem(1:100, cello_full_timeline(1:100), 'rx', 'marker', 'none', '
    LineWidth', 1.5);
```

```
31 legend('Original Signal', 'Recovered Signal (Zeros at lost indices)')
    ;
32 title('Cello Partial Recovery: Demonstration of Lost Samples');
33 xlabel('Sample Index (n)'); ylabel('Amplitude'); grid on;
34 hold off;
35
36 % =====
37 % 6d. CELLO RECONSTRUCTION (INTERPOLATION)
38 % =====
39 known_indices = (2 * (1:N_cello_max) + 1)';
40 known_values = cello_recovered_partial;
41 query_indices = (3:known_indices(end))';
42
43 % Cubic spline interpolation for smooth audio reconstruction
44 cello_interpolated = interp1(known_indices, known_values,
    query_indices, 'spline');
45
46 % --- VISUALIZATION ---
47 figure('Name', 'Cello Interpolation Result');
48 stem(query_indices(1:150), cello.x(query_indices(1:150)), 'b', ...
49     'marker', 'none', 'LineWidth', 1.5); hold on;
50 stem(query_indices(1:150), cello_interpolated(1:150), 'g', ...
51     'marker', 'none', 'LineWidth', 1.5);
52 legend('Original Cello', 'Interpolated Guess');
53 title('Cello Recovery via Spline Interpolation (First 150 samples)');
54 xlabel('Sample Index'); ylabel('Amplitude'); grid on;
55 hold off;
```

2.3.3 Results and Plots (Cello)

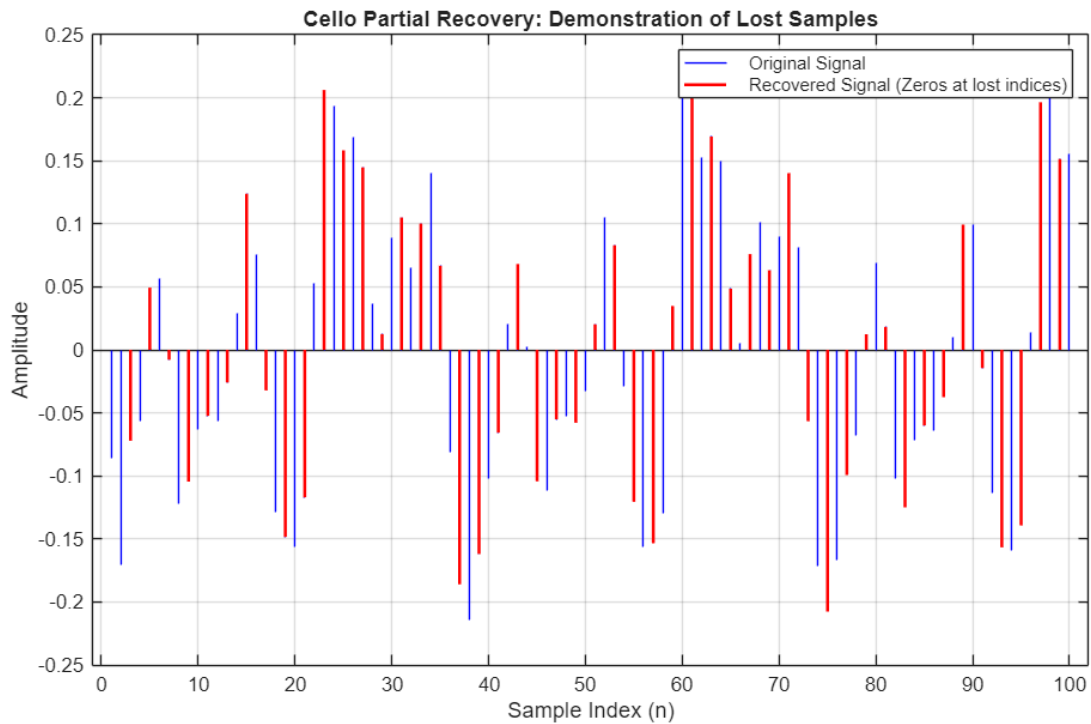


Figure 5: Cello Partial Recovery: Demonstration of Lost Samples

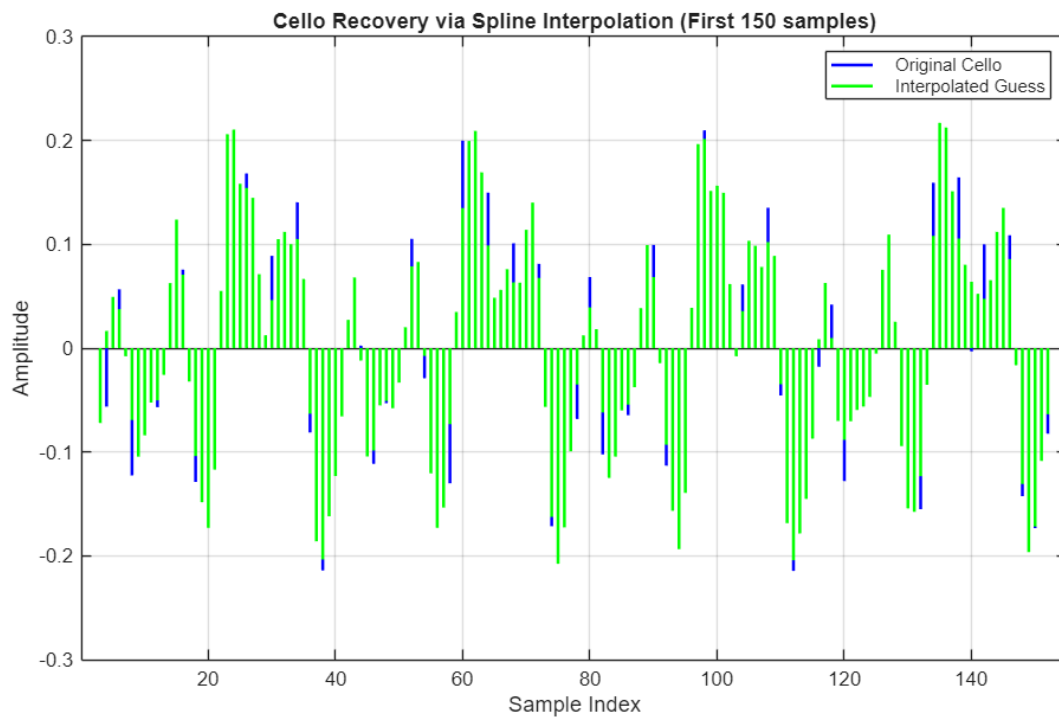


Figure 6: Cello Recovery via Spline Interpolation (First 150 Samples)

- **Partial Recovery:** As shown in Figure 5, at every odd-indexed position the recovered amplitude exactly matches the original signal, confirming that the derived inverse equation correctly inverts the scaling and DC offset. At every even-indexed position the value is zero, demonstrating the permanent information loss caused by the downsampling step of the corruption.
- **Spline Interpolation:** As shown in Figure 6, the cubic spline interpolation attempts to estimate the missing even-indexed samples from the surrounding known odd-indexed values. The interpolated signal broadly follows the envelope of the original; however, visible discrepancy may be visible at many positions. This is expected, since interpolation can only estimate information that was permanently discarded by the corruption. The accuracy of the estimate is limited by how rapidly the signal varies between adjacent known samples.

2.3.4 Trombone: Corruption and Full Recovery

The trombone corrupting system is a causal 2-point moving-average filter:

$$y_t[n] = \frac{1}{2}(x_t[n] + x_t[n-1]), \quad n \geq 2$$

with the initial condition $y_t[1] = \frac{1}{2}x_t[1]$, implying $x_t[0] = 0$. Unlike the cello case, no samples are discarded: every sample of $x_t[n]$ contributes to the output $y_t[n]$, so the original signal can be recovered. Rearranging for $x_t[n]$ yields the inverse difference equation:

$$x_t[n] = 2y_t[n] - x_t[n-1], \quad n \geq 2$$

with initial condition $x_t[1] = 2y_t[1]$. Here, each recovered sample depends on the current corrupted sample and the previously recovered sample.

2.3.5 Code (Trombone)

```
1 % =====
2 % 7. TROMBONE CORRUPTION & FULL RECOVERY
```

```
3 % =====
4 % Formula: TromboneCorrupted[n] = 0.5*(trombone[n] + trombone[n-1])
5 trombone_corrupted = zeros(length(trombone.x), 1);
6
7 % 7a. Simulate corruption
8 trombone_corrupted(1) = 0.5 * trombone.x(1);
9 for n = 2:length(trombone.x)
10     trombone_corrupted(n) = 0.5 * (trombone.x(n) + trombone.x(n-1));
11 end
12
13 % 7b & 7c. Trombone Full Recovery
14 % Inverse difference equation:  $x[n] = 2*y[n] - x[n-1]$ 
15 trombone_recovered = zeros(length(trombone_corrupted), 1);
16 trombone_recovered(1) = 2 * trombone_corrupted(1);
17 for n = 2:length(trombone_corrupted)
18     trombone_recovered(n) = 2 * trombone_corrupted(n) -
19         trombone_recovered(n-1);
20 end
21 % Verify recovery success
22 figure('Name', 'Trombone Recovery Verification');
23 plot(1:100, trombone.x(1:100), 'b', 'LineWidth', 2); hold on;
24 plot(1:100, trombone_recovered(1:100), 'r--', 'LineWidth', 1.5);
25 legend('Original', 'Recovered');
26 title('Verification of Complete Trombone Recovery');
27 xlabel('Sample Index (n)'); ylabel('Amplitude'); grid on;
28 hold off;
```

2.3.6 Results and Plots (Trombone)

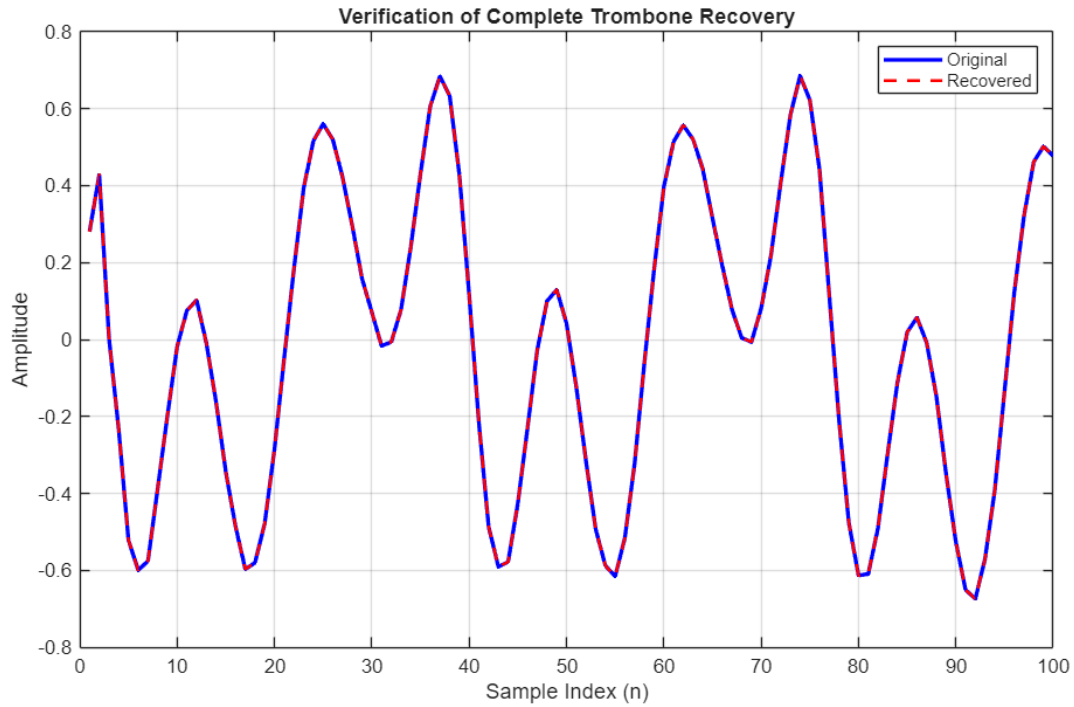


Figure 7: Verification of Complete Trombone Recovery

- **Full Recovery:** As observed in Figure 7, the recovered trombone signal (red dashed) is completely indistinguishable from the original (blue solid) across all 100 plotted samples. This confirms that the inverse difference equation $x_t[n] = 2y_t[n] - x_t[n - 1]$ achieves exact recovery, as expected from the fact that no information was lost during the moving-average corruption.
- **Comparison with Cello:** The trombone corruption is a Linear Time-Invariant (LTI) system that preserves all sample information, making exact inversion possible. In contrast, the cello corruption involves an irreversible downsampling step, limiting recovery to a partial reconstruction.